

Rapports de Travaux Pratiques

Module : Maquette et Ingénierie Numérique

TP 1 —Modélisation UML et Gestion de Versions

TP 2 —Système Solaire sous Unity 3D

TP 3 —Gestion de Versions avec Git

Réalisé par : Saad Benhaimer

Encadrant : Pr. Hrimech Hamid

Année universitaire : 2025/2026

Table des matières

1	Introduction	2
2	Cas 1 : Plateforme E-commerce	2
2.1	Description	2
2.2	Diagramme de cas d'utilisation	2
2.3	Diagramme de séquence	4
2.4	Diagramme d'activités	4
2.5	Gestion de versions avec Git	6
2.5.1	Initialisation du dépôt	6
2.5.2	Création des branches de fonctionnalités	6
2.5.3	Développement d'une fonctionnalité	6
2.5.4	Fusion dans la branche develop	6
3	Cas 2 : Gestion Universitaire	7
3.1	Description	7
3.2	Travail collaboratif	8
3.3	Workflow Collaboratif Git	8
3.3.1	Création des branches	8
3.3.2	Développement et Push	8
3.3.3	Pull Request	8
3.3.4	Mise à jour locale	9
4	Cas 3 : Gestion des conflits Git	10
4.1	Objectif	10
4.2	Simulation complète	10
4.2.1	Création du dépôt	10
4.2.2	Développeur 1	10
4.2.3	Développeur 2	10
4.2.4	Fusion provoquant le conflit	10
4.3	Conflit obtenu	10
4.4	Résolution du conflit	11
5	Cas 4 : Système de Réservation d'Hôtel	12
5.1	Description	12
6	Conclusion	14

1 Introduction

Ce projet présente la modélisation UML et la gestion de versions Git pour différents cas d'étude, permettant d'allier la conception logicielle à une méthodologie de développement collaborative et rigoureuse.

2 Cas 1 : Plateforme E-commerce

2.1 Description

La plateforme permet aux clients de consulter des produits, de gérer leur panier et d'effectuer des paiements de manière sécurisée.

FIGURE 1 – Architecture globale de la plateforme E-commerce

2.2 Diagramme de cas d'utilisation

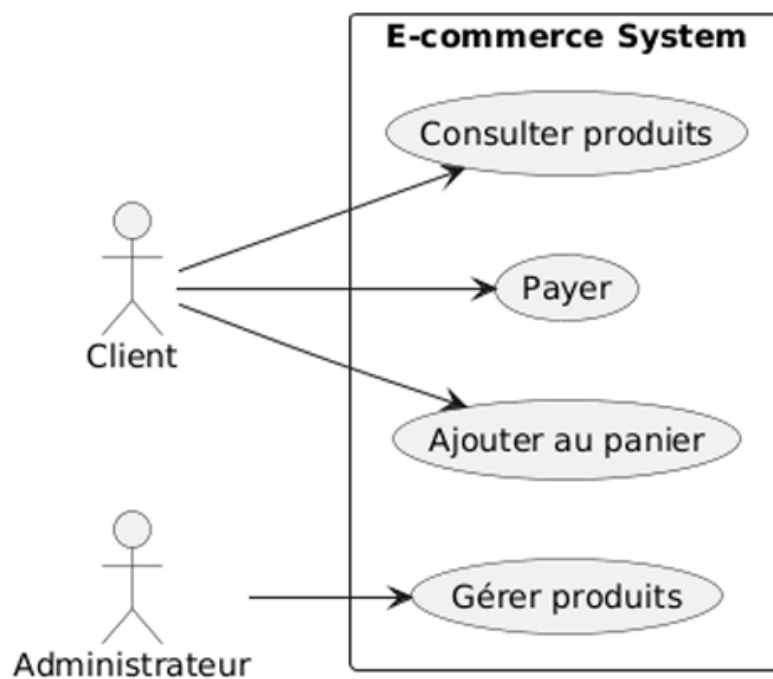


FIGURE 2 – Diagramme de cas d'utilisation

FIGURE 3 – Diagramme de cas d'utilisation - E-commerce

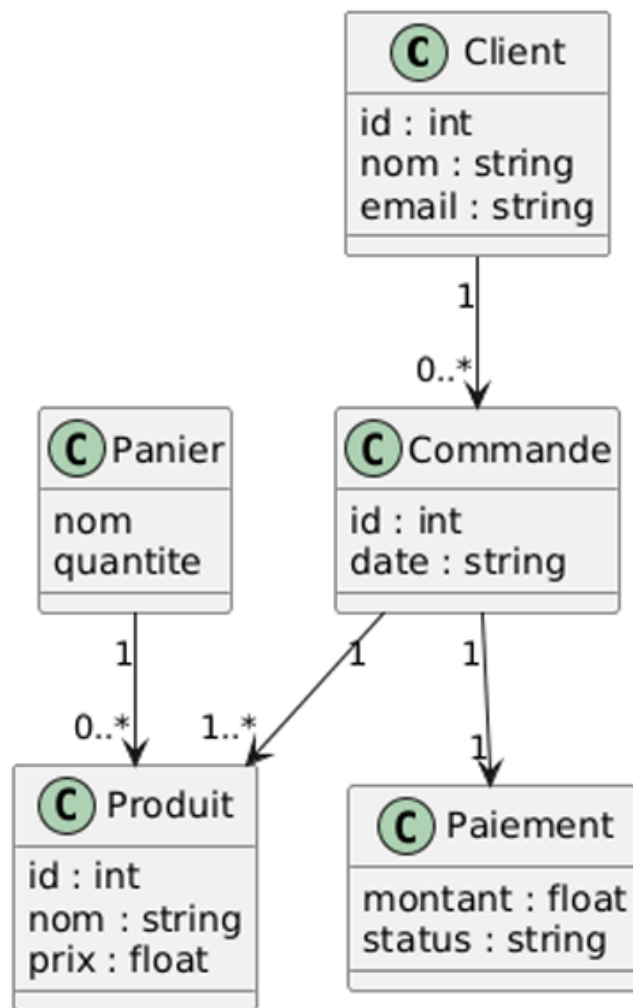


FIGURE 4 – Diagramme de classe

2.3 Diagramme de séquence

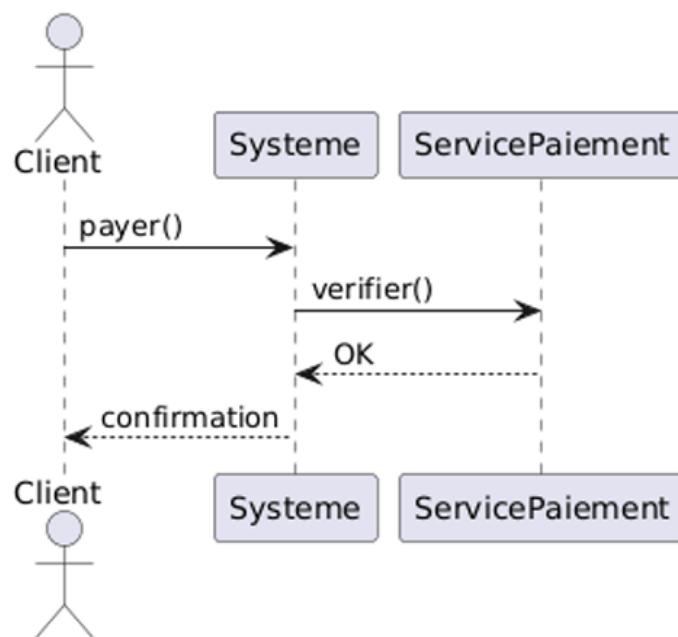


FIGURE 5 – Diagramme de sequence

2.4 Diagramme d'activités

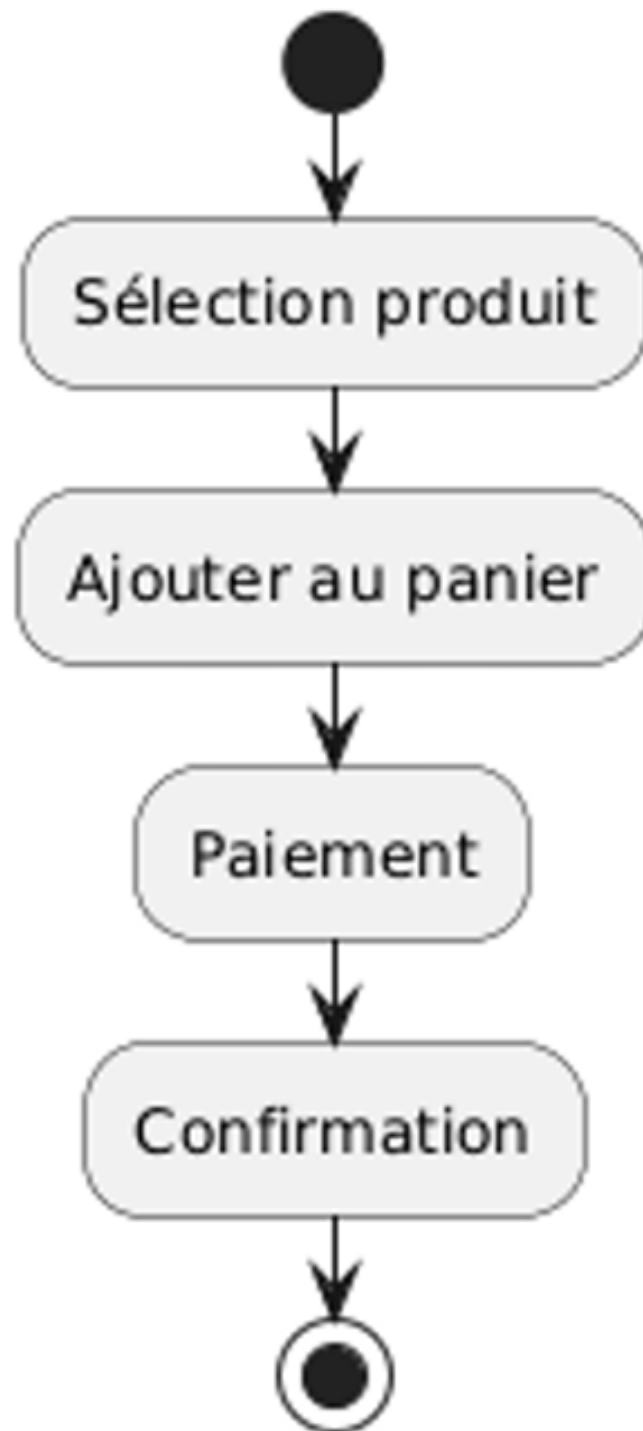


FIGURE 6 – Diagramme d'activités

2.5 Gestion de versions avec Git

Le cycle de développement des fonctionnalités de la plateforme a été orchestré à l'aide de Git, en suivant le workflow standard Git Flow (branches de fonctionnalités dérivées de **develop**).

2.5.1 Initialisation du dépôt

L'initialisation et la configuration initiale du dépôt principal s'effectuent ainsi :

```
1 git init
2 git add .
3 git commit -m "Initial commit with UML diagrams"
4 git branch -M main
5 git checkout -b develop
```

2.5.2 Création des branches de fonctionnalités

Chaque module de la plateforme fait l'objet d'une branche isolée pour garantir la stabilité de la branche de développement :

```
1 git checkout -b feature/produits
2 git checkout develop
3 git checkout -b feature/panier
4 git checkout develop
5 git checkout -b feature/paiement
6 git checkout develop
```

2.5.3 Développement d'une fonctionnalité

Exemple d'implémentation et de publication des modifications pour le module de paiement :

```
1 git checkout feature/paiement
2 echo "Fonction paiement implémenté" > paiement.txt
3 git add paiement.txt
4 git commit -m "Implémentation paiement"
5 git push origin feature/paiement
```

2.5.4 Fusion dans la branche develop

Une fois la fonctionnalité validée, elle est intégrée à la branche de développement commune :

```
1 git checkout develop
2 git merge feature/paiement
3 git push origin develop
```

3 Cas 2 : Gestion Universitaire

3.1 Description

Cette application permet d'assurer le suivi et la gestion administrative des étudiants, des enseignants ainsi que l'affectation et la planification des différents cours.

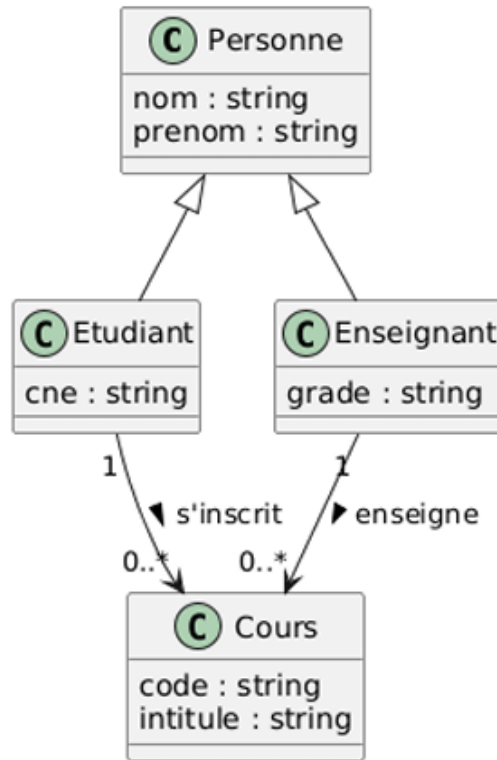


FIGURE 7 – Diagramm de Classe

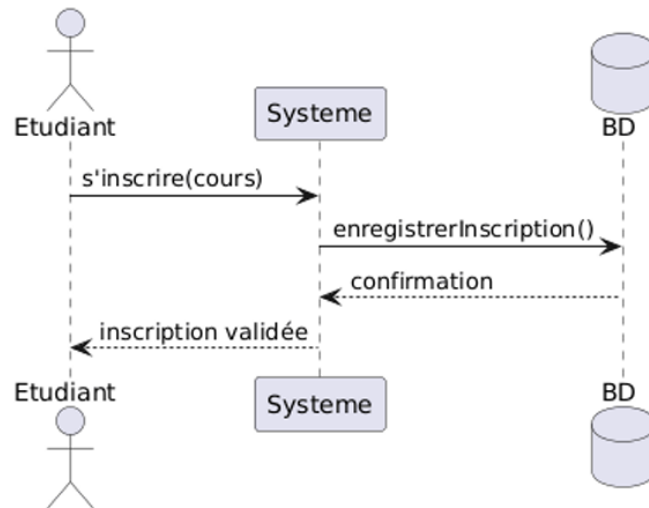


FIGURE 8 – Diagramme de séquence

3.2 Travail collaboratif

L'organisation du travail en équipe repose sur des principes stricts de gestion de flux de production :

- Création de branches **feature/*** dédiées pour chaque tâche isolée.
- Synchronisation régulière via des **push** vers le dépôt distant GitHub.
- Soumission obligatoire d'une **Pull Request** (PR) avant toute intégration.
- Validation croisée par les pairs puis fusion finale.

3.3 Workflow Collaboratif Git

3.3.1 Création des branches

Chaque membre de l'équipe initialise sa branche de travail locale à partir de la dernière version stable :

```

1 git checkout -b feature/etudiant
2 git checkout -b feature/enseignant
3 git checkout -b feature/cours

```

3.3.2 Développement et Push

Une fois la modification apportée localement, les fichiers sont indexés, archivés et poussés sur le serveur distant :

```

1 echo "Gestion Etudiant" > etudiant.txt
2 git add .
3 git commit -m "Ajout gestion tudiant "
4 git push origin feature/etudiant

```

3.3.3 Pull Request

Après le push vers GitHub :

- Création d'une **Pull Request** ciblant la branche **develop**.

- Phase d’inspection et de validation du code.
- Fusion (*Merge*) dans la branche **develop**.
- Suppression de la branche de fonctionnalité devenue obsolète.

3.3.4 Mise à jour locale

Pour entamer un nouveau cycle de développement, la base locale doit être resynchronisée :

```
1 git checkout develop
2 git pull origin develop
```

4 Cas 3 : Gestion des conflits Git

4.1 Objectif

Comprendre l'origine, le mécanisme de détection et la méthodologie de résolution manuelle des conflits lors de fusions concurrentes sous Git.

4.2 Simulation complète

4.2.1 Création du dépôt

```
1 git init
2 echo "Contenu initial" > fichier.txt
3 git add .
4 git commit -m "Initial commit"
```

4.2.2 Développeur 1

Le premier développeur modifie la ligne sur sa propre branche alternative :

```
1 git checkout -b dev1
2 echo "Version A de Dev1" > fichier.txt
3 git add fichier.txt
4 git commit -m "Modification par Dev1"
```

4.2.3 Développeur 2

Simultanément, le second développeur produit une modification divergente sur la même ligne depuis la souche initiale :

```
1 git checkout main
2 git checkout -b dev2
3 echo "Version B de Dev2" > fichier.txt
4 git add fichier.txt
5 git commit -m "Modification par Dev2"
```

4.2.4 Fusion provoquant le conflit

La collision survient immédiatement lorsque l'on tente d'unifier les deux historiques divergents sans hiérarchie claire :

```
1 git checkout dev1
2 git merge dev2
```

4.3 Conflit obtenu

Lors de la fusion de la branche `dev2` dans la branche `dev1`, Git détecte que les deux développeurs ont modifié la même ligne du fichier `fichier.txt`. Le système ne peut pas déterminer automatiquement quelle version conserver et génère alors un conflit d'intégration.

Le contenu du fichier se transforme pour inclure les balises de conflit :

```
1 <<<<<< HEAD
2 Version A de Dev1
3 =====
4 Version B de Dev2
5 >>>>>> dev2
```

Les marqueurs affichés par Git permettent d'identifier précisément la provenance des lignes en opposition :

- <<<<<< HEAD : version de la branche courante (**dev1**).
- ===== : séparateur strict entre les deux versions concurrentes.
- >>>>>> dev2 : version provenant de la branche entrante (**dev2**).

Pour résoudre le conflit, le développeur doit modifier manuellement le fichier, supprimer l'intégralité de ces marqueurs techniques et arbitrer pour la version finale souhaitée.

4.4 Résolution du conflit

Après modification manuelle du fichier `fichier.txt` pour obtenir le résultat attendu, la finalisation de la fusion s'exécute ainsi :

```
1 git add fichier.txt
2 git commit -m "R é s o l u t i o n   d u   c o n f l i t"
```

5 Cas 4 : Système de Réservation d'Hôtel

5.1 Description

Application permettant la réservation de chambres d'hôtel, incluant le suivi des disponibilités, des profils clients et de la facturation associée.

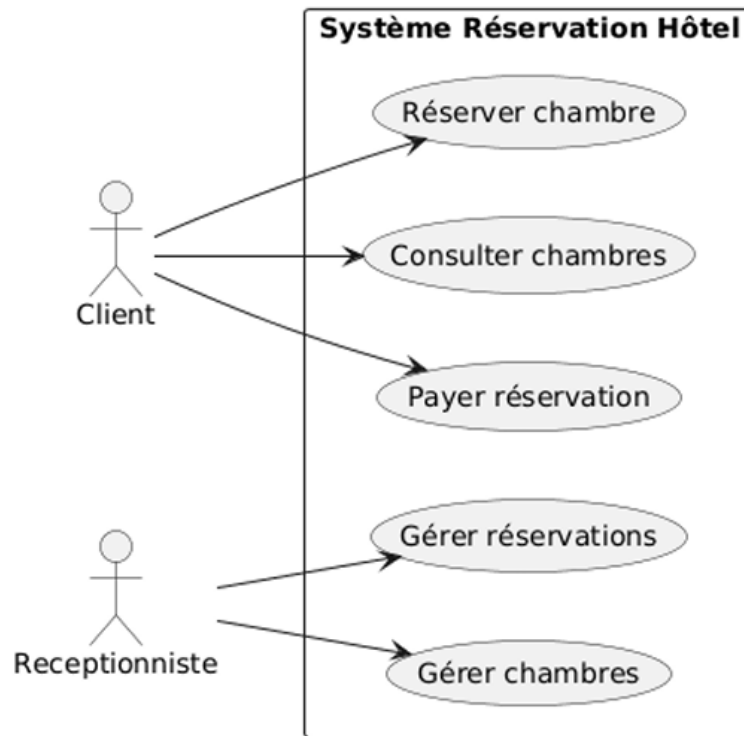


FIGURE 9 – Use case

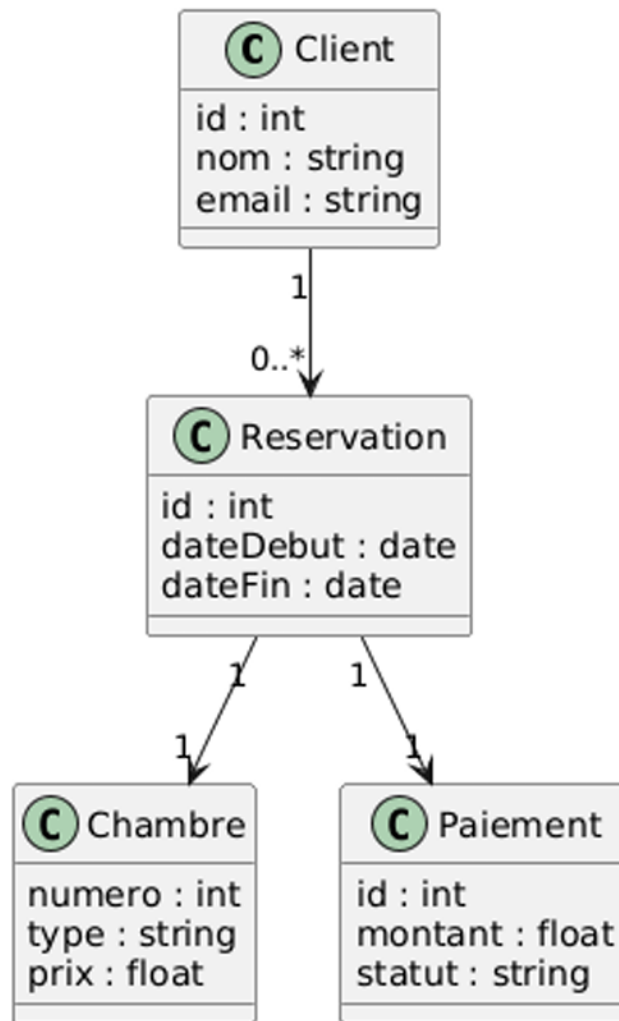


FIGURE 10 – Class

6 Conclusion

En somme, ce projet a permis de concrétiser l’alliance indispensable entre la modélisation conceptuelle avec UML et la gestion de configuration logicielle via Git et GitHub.

À travers l’analyse des différents cas d’étude, l’élaboration des diagrammes (cas d’utilisation, de classes et de séquence) a offert une vision claire et structurée du comportement architectural des systèmes. Parallèlement, l’application concrète des workflows Git (GitFlow, gestion des branches, Pull Requests) et la résolution manuelle de conflits de fusion ont fourni les clés d’une collaboration d’équipe efficace et sécurisée.

Cette approche globale nous offre une immersion concrète dans les méthodologies industrielles actuelles. Elle démontre qu’une conception logicielle rigoureuse, associée à une gestion de version maîtrisée, constitue le socle indispensable à la réussite, à la qualité et à la maintenabilité de tout projet d’ingénierie numérique.

Table des matières

1	Création de la Scène	4
1.1	Création du Soleil	4
1.1.1	Paramètres du Soleil	4
1.1.2	Application de la Texture	5
1.2	Ajout des Planètes	6
1.3	Configuration du Skybox	7
2	Animation et Scripts	8
2.1	Principe d'Animation	8
2.2	Script d'Animation des Planètes	8
3	Contrôle de la Caméra	10
3.1	Script MouseAimCamera	10
3.2	Paramètres Importants	12

Résumé détaillé du TP – Système Solaire sous Unity 3D

Objectif général

Ce travail pratique a pour vocation la conception et l'animation d'une scène 3D interactive représentant le système solaire héliocentrique. L'apprenant doit modéliser le Soleil et les huit planètes, leur appliquer des textures réalistes, les animer par rotation propre et révolution orbitale, et naviguer dans la scène à l'aide d'une caméra contrôlée par la souris.

Compétences visées

À l'issue de ce TP, l'étudiant sera capable de :

- Manipuler les transformations spatiales (`Translate`, `Rotate`, `RotateAround`) dans un environnement 3D ;
- Importer et appliquer des textures sur des maillages sphériques ;
- Configurer un *Skybox* pour l'immersion environnementale ;
- Écrire des scripts C# pour l'animation temps réel via la méthode `Update()` ;
- Implémenter un système de caméra orbitale à la souris (*Mouse Aim*) ;
- Gérer l'arborescence des assets (dossiers `Textures`, `Materials`, `Audio`, `Scripts`).

Étapes de réalisation

Étape 1 – Importation des ressources

- Récupération des textures planétaires et solaires (<https://www.solarsystemscope.com/textures/>) ;
- Importation de l'asset *Starfield Skybox* pour l'arrière-plan spatial.

Étape 2 – Construction de la scène

- Création d'une sphère nommée « Soleil », positionnée à l'origine (0,0,0) et mise à l'échelle ;
- Création des huit planètes (Mercure, Vénus, Terre, Mars, Jupiter, Saturne, Uranus, Neptune) sous forme de sphères ;
- Organisation hiérarchique : création d'un dossier `Texture` contenant un sous-dossier `Material` pour l'application des textures ;
- Assignation du *Skybox* à la caméra via le *Rendering Settings*.

Étape 3 – Programmation des animations (C#) Deux comportements distincts sont implémentés pour chaque planète :

1. Rotation propre :

```
transform.Rotate(0, 35 * Time.deltaTime, 0);
```

2. Révolution orbitale autour du Soleil :

```
GameObject soleil = GameObject.Find("Soleil");  
transform.RotateAround(soleil.transform.position,  
                        -soleil.transform.up,
```

`70 * Time.deltaTime);`

Étape 4 – Contrôle de la caméra Développement d'un script `MouseAimCamera` permettant :

- La rotation orbitale de la caméra autour d'une cible (Soleil ou planète) selon l'axe horizontal de la souris (`Mouse X`) ;
- Le maintien d'un décalage (*offset*) constant entre la caméra et la cible ;
- L'orientation continue de la caméra vers la cible via `transform.LookAt()` ;
- L'extension du script pour intégrer le contrôle sur les axes **X et Y**.

Étape 5 – Enrichissement multimédia

- Création d'un dossier `Audio` ;
- Intégration d'une ambiance sonore spatiale (optionnel selon le fichier audio fourni).

Contraintes et points d'attention

- Le script de caméra doit être attaché à la `Main Camera` avec la cible (`target`) correctement référencée dans l'inspecteur ;
- La méthode `LateUpdate()` est privilégiée à `Update()` pour le contrôle caméra afin d'éviter les décalages de rendu ;
- Les vitesses de rotation et de révolution doivent être multipliées par `Time.deltaTime` pour garantir une animation indépendante du framerate.

Chapitre 1

Création de la Scène

1.1 Création du Soleil

Le Soleil constitue le point central de notre système solaire miniature, autour duquel gravitent tous les autres astres.

1.1.1 Paramètres du Soleil

Pour initialiser le Soleil, les étapes suivantes ont été appliquées dans l'interface d'Unity :

1. Insertion d'une sphère dans la scène (`GameObject > 3D Object > Sphere`).
2. Renommage de l'objet en « `Soleil` ».
3. Ajustement de la taille via la composante *Transform* (Scale réglé proportionnellement).
4. Positionnement rigoureux à l'origine absolue de la scène : $(0, 0, 0)$.

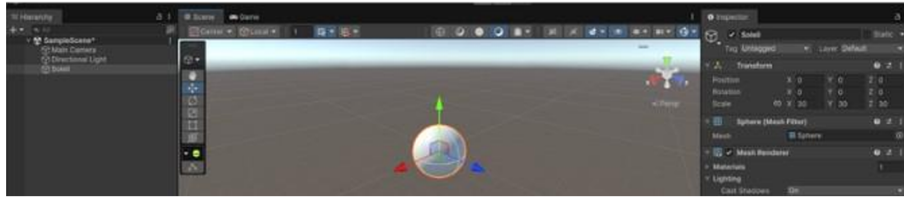


FIGURE 1.1 – Enter Caption

1.1.2 Application de la Texture

Afin de donner au Soleil son aspect lumineux et réaliste, une texture émissive spécifique a été importée et assignée à son *Material*. Les paramètres d'albedo et d'émission ont été configurés pour simuler le rayonnement de l'étoile.

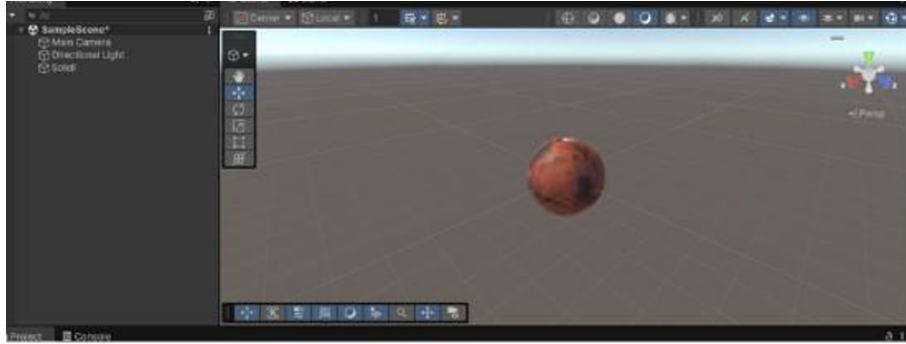


FIGURE 1.2 – Enter Caption

1.2 Ajout des Planètes

Suivant une logique similaire, les différentes planètes du système ont été générées sous forme de sphères à des distances croissantes de l'origine. Chacune possède des dimensions ajustées à sa nature ainsi qu'une texture dédiée mappée à son matériau standard pour les différencier visuellement.



FIGURE 1.3 – Enter Caption

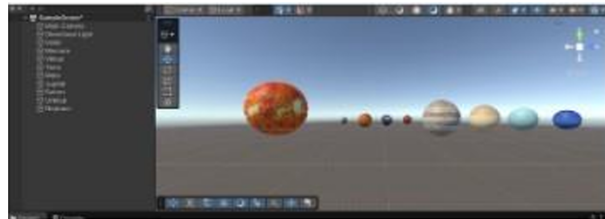


FIGURE 1.4 – Enter Caption

1.3 Configuration du Skybox

Pour immerger le système solaire dans un environnement spatial réaliste, le *Skybox* par défaut d'Unity a été remplacé. Un nouveau matériau de type *Skybox/6 Sided* a été créé, utilisant des textures de voûte céleste étoilée appliquées sur ses six faces internes.



FIGURE 1.5 – Enter Caption

Chapitre 2

Animation et Scripts

2.1 Principe d'Animation

Les mouvements au sein du système solaire reposent sur deux principes physiques distincts implémentés par programmation :

- **La Rotation (Self-Rotation)** : Le mouvement de l'astre sur lui-même autour de son axe vertical.
- **La Révolution (Orbit)** : Le déplacement orbital de la planète autour du Soleil central.

2.2 Script d'Animation des Planètes

Voici le script C# conçu pour gérer simultanément la rotation interne et la révolution orbitale de chaque planète :

```
1 using UnityEngine;
2
3 public class OrbitAndRotate : MonoBehaviour
4 {
5     [Header("Parametres de Revolution")]
6     public Transform centerObject; // Le Soleil
7     public float orbitSpeed = 20f;
8
9     [Header("Parametres de Rotation")]
10    public float rotationSpeed = 30f;
11
12    void Update()
13    {
14        // 1. Gestion de la Revolution (Orbite) autour du centre
15        if (centerObject != null)
16        {
17            transform.RotateAround(centerObject.position, Vector3.
18            up, orbitSpeed * Time.deltaTime);
19
20            // 2. Gestion de la Rotation de la planete sur elle-meme
21            transform.Rotate(Vector3.up, rotationSpeed * Time.deltaTime
22            );
23        }
24    }
25 }
```

```
22     }  
23 }
```

Listing 2.1 – Script d’animation OrbitAndRotate.cs

Chapitre 3

Contrôle de la Caméra

3.1 Script MouseAimCamera

Pour permettre à l'utilisateur de naviguer dynamiquement autour du système solaire en cours d'exécution, un script de contrôle de caméra basé sur les entrées de la souris a été implémenté.

```
1 using UnityEngine;
2
3 public class MouseAimCamera : MonoBehaviour
4 {
5     public Transform target; // L'objet à observer
6     public float distance = 10.0f;
7     public float xSpeed = 120.0f;
8     public float ySpeed = 120.0f;
9
10    public float yMinLimit = -20f;
11    public float yMaxLimit = 80f;
12
13    private float x = 0.0f;
14    private float y = 0.0f;
15
16    void Start()
17    {
18        Vector3 angles = transform.eulerAngles;
19        x = angles.y;
20        y = angles.x;
21
22        if (GetComponent<Rigidbody>())
23        {
24            GetComponent<Rigidbody>().freezeRotation = true;
25        }
26    }
27
28    void LateUpdate()
29    {
30        if (target && Input.GetMouseButton(1)) // Clic droit pour
orbiter
31        {
```

```

32         x += Input.GetAxis("Mouse X") * xSpeed * distance *
0.02f;
33         y -= Input.GetAxis("Mouse Y") * ySpeed * 0.02f;
34
35         y = ClampAngle(y, yMinLimit, yMaxLimit);
36
37         Quaternion rotation = Quaternion.Euler(y, x, 0);
38         Vector3 negDistance = new Vector3(0.0f, 0.0f, -distance
);
39         Vector3 position = rotation * negDistance + target.
position;
40
41         transform.rotation = rotation;
42         transform.position = position;
43     }
44 }
45
46 public static float ClampAngle(float angle, float min, float
max)
47 {
48     if (angle < -360F) angle += 360F;
49     if (angle > 360F) angle -= 360F;
50     return Mathf.Clamp(angle, min, max);
51 }
52 }

```

Listing 3.1 – Script de controle camera MouseAimCamera.cs

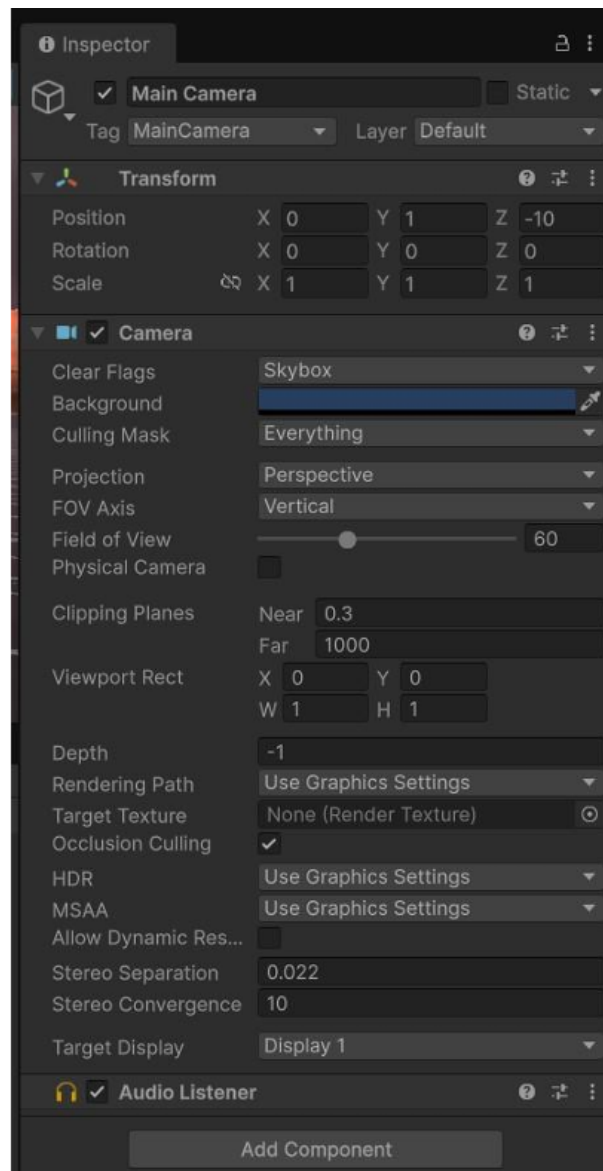


FIGURE 3.2 – Paramètres du script dans l'Inspector

Conclusion

Ce travail pratique a permis de modéliser une simulation interactive du système solaire sous Unity 3D, en combinant à la fois des aspects graphiques, algorithmiques et ergonomiques. À travers la création et l’animation des corps célestes, nous avons mis en œuvre les principes fondamentaux de transformation spatiale en trois dimensions — translation, rotation et orbite — tout en exploitant le rendu temps réel du moteur de jeu.

L’intégration de textures réalistes et d’un *Skybox* spatial a sensiblement amélioré l’immersion visuelle de la scène, tandis que la programmation en C# a offert un contrôle précis et paramétrable des trajectoires planétaires. Par ailleurs, l’implémentation d’une caméra contrôlée par la souris a enrichi l’interactivité du projet, permettant une exploration libre et intuitive de l’environnement modélisé.

Au-delà du résultat visuel, ce TP a constitué une initiation concrète au développement de scènes 3D temps réel, compétence transposable à de nombreux domaines de l’ingénierie des systèmes d’information : visualisation de données, simulation scientifique, réalité virtuelle ou encore conception de serious games. La maîtrise de ces outils et méthodologies ouvre ainsi la voie à des projets plus ambitieux alliant informatique graphique et interactivité utilisateur.

Table des matières

1	Introduction	2
2	Initialisation et configuration d'un dépôt local	2
2.1	Création de l'architecture du projet	2
2.2	Initialisation du dépôt	2
2.3	Premier commit	2
3	Gestion des branches	2
3.1	Affichage des branches existantes	2
3.2	Création et basculement de branches	2
3.3	Travail isolé sur une branche	3
3.4	Modification parallèle sur la branche principale	3
4	Fusion de branches (Merge)	3
4.1	Fusion sans conflit	3
4.2	Vérification post-fusion	3
5	Gestion des conflits de fusion	4
5.1	Principe du conflit	4
5.2	Simulation d'un conflit	4
5.3	Tentative de fusion et détection du conflit	4
5.4	Analyse des marqueurs de conflit	4
5.5	Résolution manuelle	4
5.6	Abandon d'une fusion	5
6	Historique et comparaison	5
6.1	Visualisation de l'historique	5
6.2	Comparaison entre branches	5
6.3	Navigation dans l'historique	5
7	Nettoyage des branches	5
8	Conclusion	5

1 Introduction

Ce rapport présente les travaux pratiques consacrés à l'apprentissage de Git, l'outil de gestion de versions distribué incontournable en développement logiciel moderne. L'objectif principal est de maîtriser la création de dépôts, la manipulation des branches, la fusion de code ainsi que la résolution des conflits de fusion.

Les compétences acquises permettent de simuler un environnement collaboratif réel, où plusieurs développeurs travaillent simultanément sur un même projet sans écraser le travail des autres.

2 Initialisation et configuration d'un dépôt local

2.1 Création de l'architecture du projet

Avant d'initialiser Git, nous avons structuré notre projet en trois répertoires distincts :

- `html/` : contient la page d'accueil (`index.html`)
- `css/` : contient la feuille de styles (`style.css`)
- `js/` : contient les scripts client (`index.js`)

2.2 Initialisation du dépôt

La commande `git init` permet de transformer un dossier ordinaire en dépôt Git traçable.

```
git init
```

Résultat : création d'un sous-dossier `.git/` contenant l'historique et la configuration du projet.

2.3 Premier commit

Après avoir créé les fichiers, nous les avons indexés puis validés :

```
git add .  
git commit -m "Initial commit : structure de base html/css/js"
```

La commande `git status` confirme alors que l'arbre de travail est propre (*nothing to commit, working tree clean*).

3 Gestion des branches

3.1 Affichage des branches existantes

```
git branch
```

Par défaut, Git crée la branche `master` (ou `main` selon la configuration). L'astérisque `*` indique la branche active.

3.2 Création et basculement de branches

Pour créer une branche dédiée au développement d'une nouvelle fonctionnalité (par exemple, une refonte visuelle) :

```
# Methode 1 : creation puis basculement
git branch style
git checkout style

# Methode 2 : creation et basculement en une seule commande
git checkout -b style
```

3.3 Travail isolé sur une branche

Sur la branche `style`, nous avons modifié le fichier `css/style.css` :

```
body {
    background-color: red;
}
```

Puis nous avons validé ces changements :

```
git add css/style.css
git commit -m "[CSS] Changement de la couleur de fond en rouge"
```

En retournant sur `master` via `git checkout master`, nous constatons que le fichier `style.css` est resté dans son état initial (bleu). Cela démontre l'isolation des branches.

3.4 Modification parallèle sur la branche principale

Toujours sur `master`, nous avons modifié `html/index.html` et committé :

```
git add html/index.html
git commit -m "[HTML] Mise a jour du contenu de la page d'accueil"
```

À ce stade, `master` et `style` possèdent des historiques divergents.

4 Fusion de branches (Merge)

4.1 Fusion sans conflit

Pour intégrer le travail de la branche `style` dans `master` :

```
git checkout master
git merge style
```

Git a effectué une fusion **fast-forward** car les modifications touchaient des fichiers différents. L'historique est devenu linéaire et contient désormais les deux évolutions.

4.2 Vérification post-fusion

Le fichier `style.css` affiche désormais le fond rouge sur `master`, confirmant que la fusion a bien intégré les modifications.

5 Gestion des conflits de fusion

5.1 Principe du conflit

Un conflit survient lorsque deux branches modifient **la même zone d'un même fichier** de manière incompatible. Git ne peut pas décider automatiquement quelle version garder.

5.2 Simulation d'un conflit

Nous avons créé un fichier `readme.md` avec le contenu initial :

```
Ligne 1 : Introduction
Ligne 2 : Description
Ligne 3 : Conclusion
```

Puis nous avons créé deux branches :

- `branch-a` : modifie la ligne 1 en ajoutant des instructions spécifiques à la branche A
- `branch-b` : modifie également la ligne 1 avec des instructions spécifiques à la branche B

5.3 Tentative de fusion et détection du conflit

```
git checkout branch-a
git merge branch-b
```

Git affiche :

```
Auto-merging readme.md
CONFLICT (content): Merge conflict in readme.md
Automatic merge failed; fix conflicts and then commit the result.
```

5.4 Analyse des marqueurs de conflit

Le fichier `readme.md` est automatiquement modifié par Git pour montrer les divergences :

```
<<<<<< HEAD
Instructions spécifiques a la branche A
=====
Instructions spécifiques a la branche B
>>>>>> branch-b
```

- `<<<< HEAD` : version de la branche courante (`branch-a`)
- `=====` : séparateur entre les deux versions
- `>>>> branch-b` : version de la branche fusionnée

5.5 Résolution manuelle

Nous avons édité le fichier pour conserver une version fusionnée cohérente :

```
Instructions fusionnees : integration des points A et B
```

Puis nous avons finalisé la fusion :

```
git add readme.md
git commit -m "Resolution du conflit entre branch-a et branch-b"
```

5.6 Abandon d'une fusion

Si une fusion est trop complexe, il est possible de l'annuler complètement :

```
git merge --abort
```

Cette commande restaure le dépôt à son état antérieur au lancement du merge.

6 Historique et comparaison

6.1 Visualisation de l'historique

```
git log --oneline --graph --all
```

Cette commande affiche l'arbre des commits avec les branches et les points de fusion.

6.2 Comparaison entre branches

```
git diff branch-a branch-b
```

Permet d'observer les différences de contenu avant de fusionner.

6.3 Navigation dans l'historique

Il est possible de consulter un ancien commit via son hash :

```
git checkout 8523ff4
```

Le détachement de HEAD permet d'inspecter le code à un instant précis sans affecter les branches actives.

7 Nettoyage des branches

Après fusion, les branches temporaires peuvent être supprimées :

```
git branch -d branch-a  
git branch -d branch-b  
git branch -d style
```

8 Conclusion

Ce TP nous a permis d'acquérir une maîtrise opérationnelle de Git dans un contexte de développement collaboratif. Nous avons appris à :

- Initialiser et structurer un dépôt
- Créer et naviguer entre des branches isolées
- Fusionner des historiques divergents
- Identifier et résoudre manuellement des conflits de fusion
- Utiliser l'historique comme outil de traçabilité

Ces compétences constituent la base indispensable pour tout projet logiciel moderne utilisant un workflow de type Git Flow ou GitHub Flow.